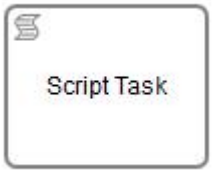
Script Task

A Script Task is an automated activity. When a process execution arrives at the Script Task, the corresponding script is executed.



A Script Task is defined by specifying the script and the Script Format.

Supported script formats are Groovy, JavaScript, JRuby and Jython.

All process variables can be accessible through the execution.

execution.getVariable("variable name");

It's also possible to set process variables in a script.

execution.setVariable("variable name","value");

Information can be accessed through execution's default variable.

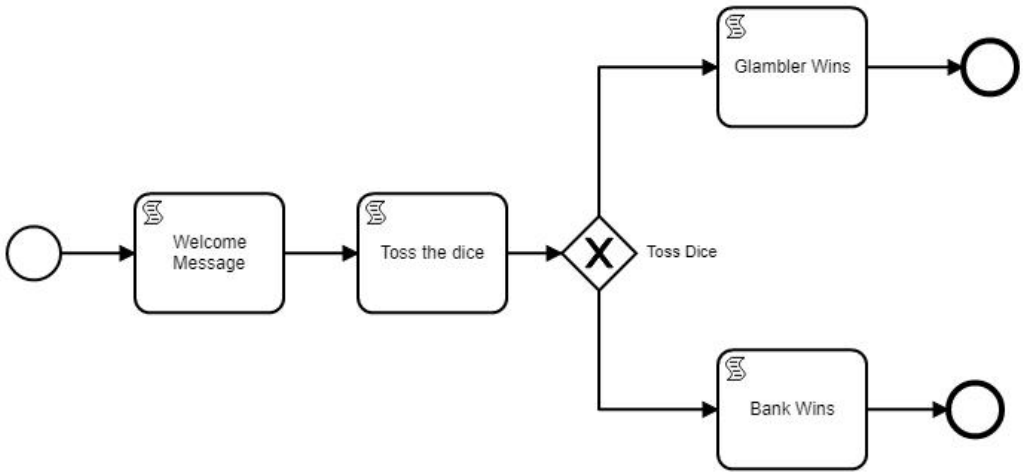
execution.eventName

execution.activityId

task.name

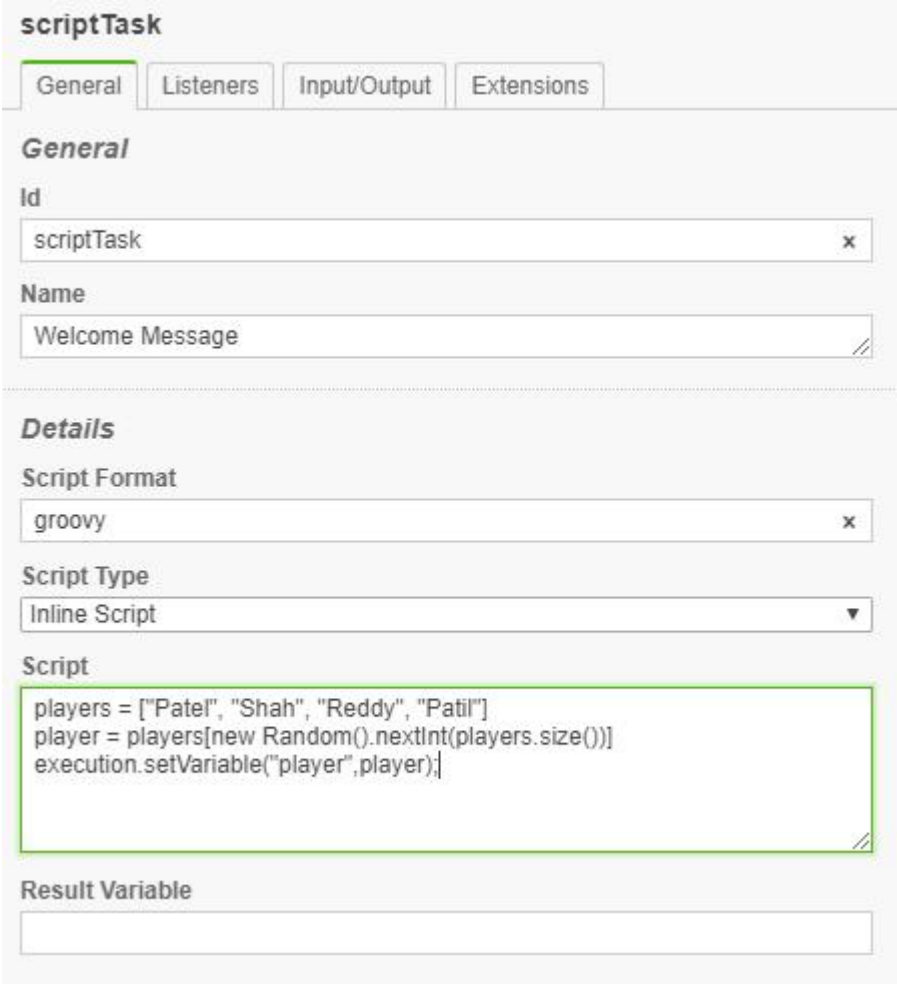
task.eventName

Exercise:



Setting properties for script each task:

Script task: Welcome Message



Script Format: groovy

Script Type: Inline Script

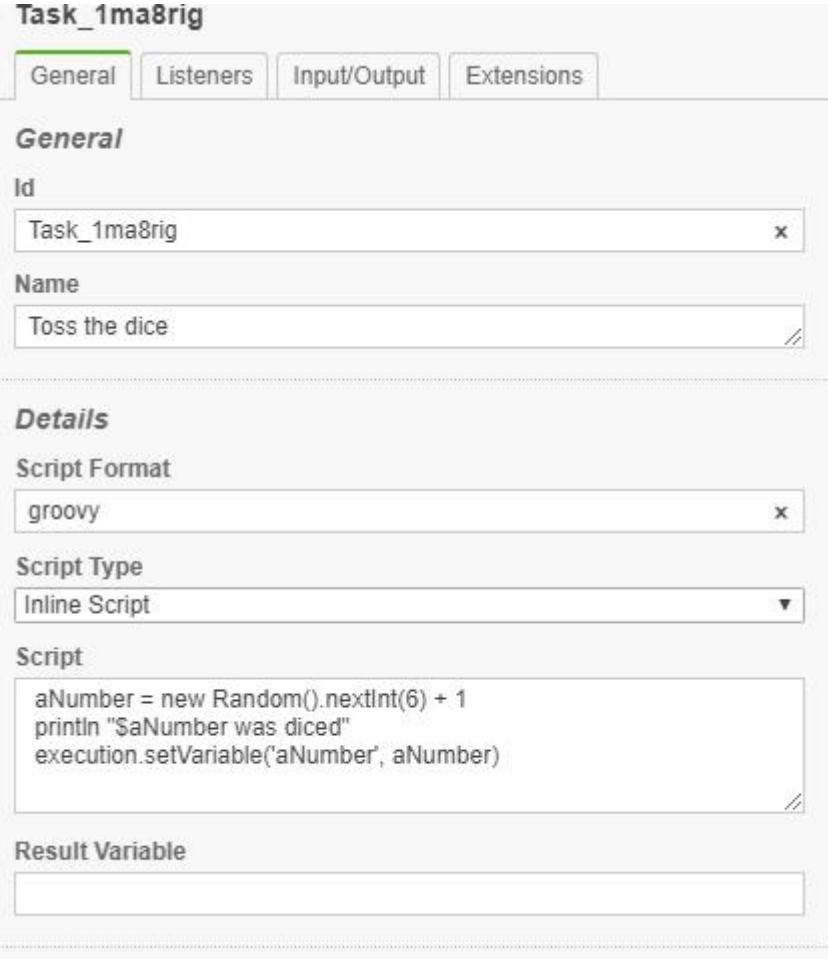
Script:

```
players = ["Patel", "Shah", "Reddy", "Patil"]

player = players[new Random().nextInt(players.size())]

execution.setVariable("player",player);
```

Script task: Toss the dice



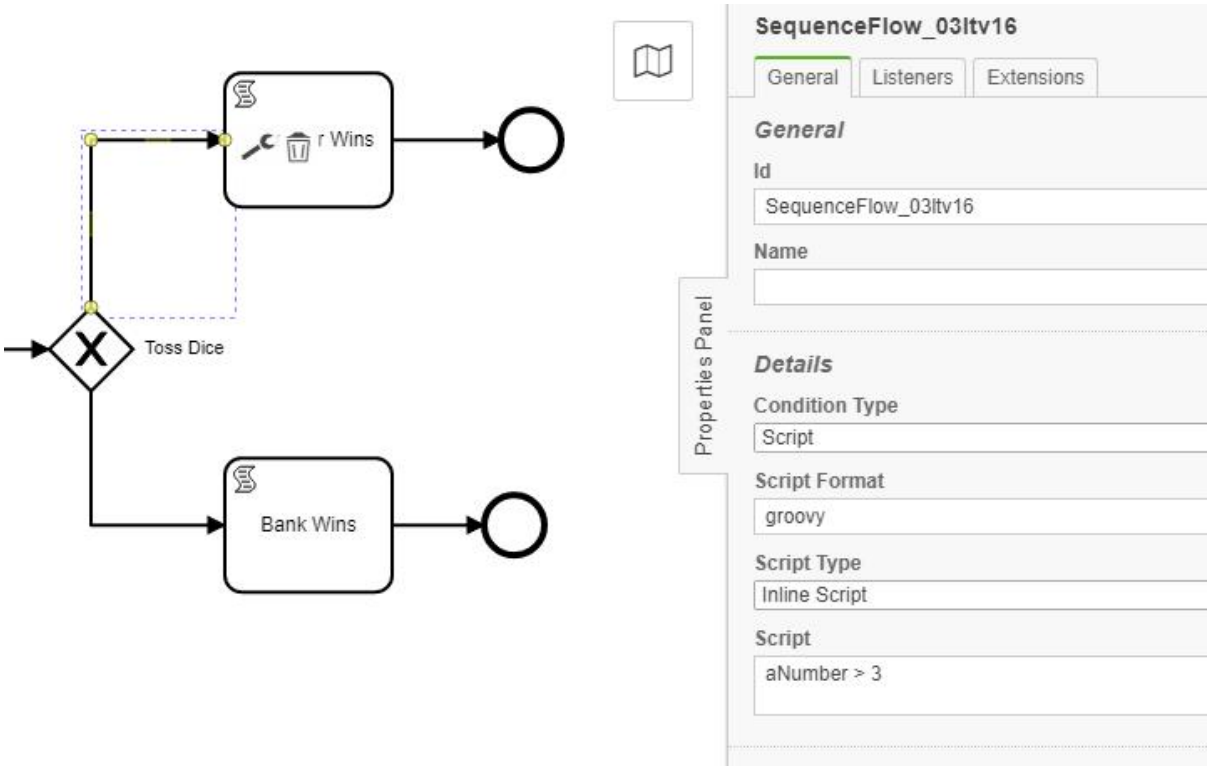
Script Format: groovy

Script Type: Inline Script

Script:

```
aNumber = new Random().nextInt(6) + 1
 println "$aNumber was diced"
 execution.setVariable('aNumber', aNumber)
```

Sequence Flow 1:

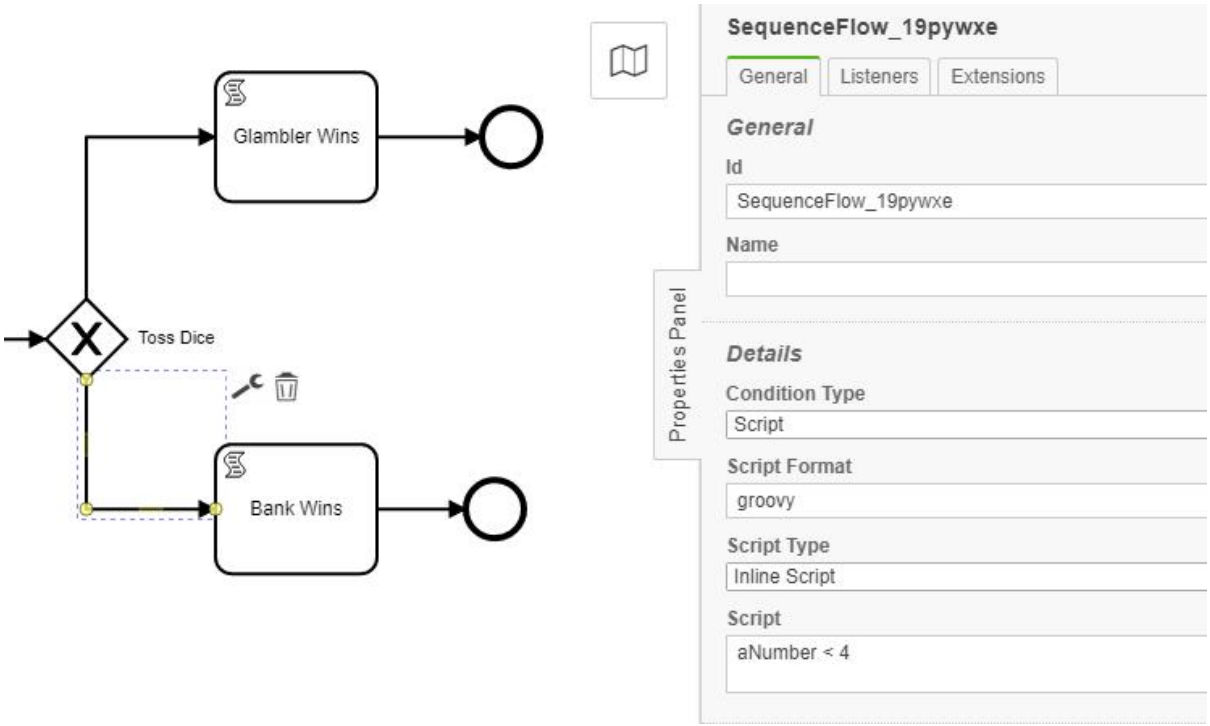


Script Format: groovy

Script Type: Inline Script

Script: aNumber > 3

Sequence Flow 2:

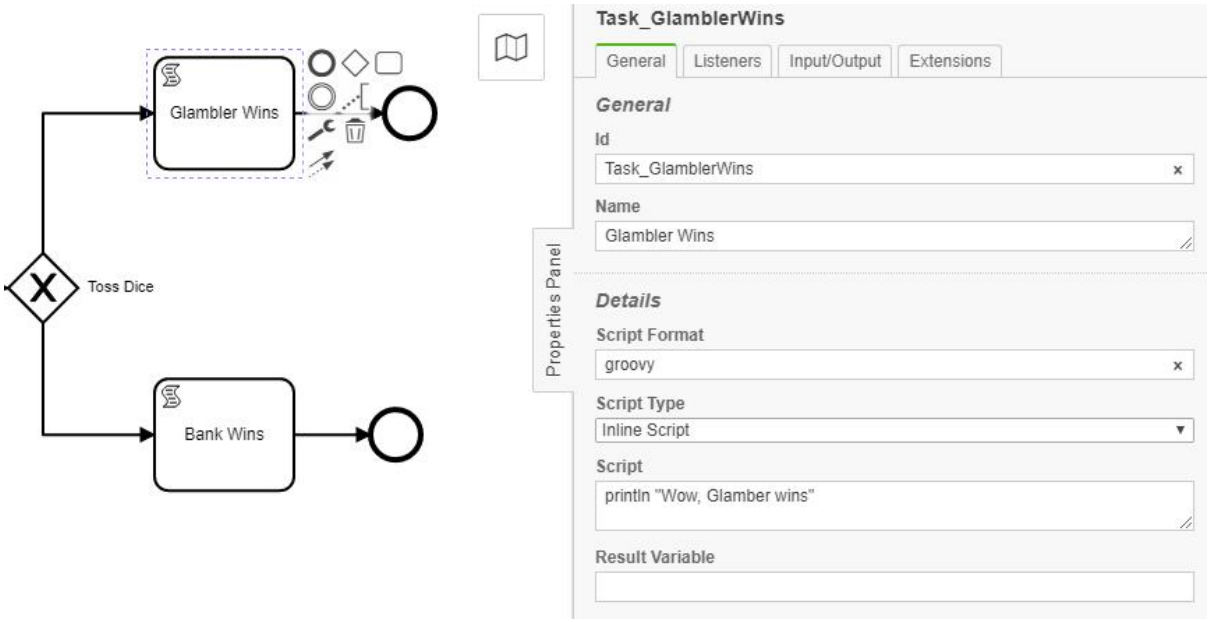


Script Format: groovy

Script Type: Inline Script

Script: aNumber < 3

Script task: Glambler Wins



Script Format: groovy

Script Type: Inline Script

Script:

```
println "Wow, Glamber wins"
```

Script task: Bank Wins

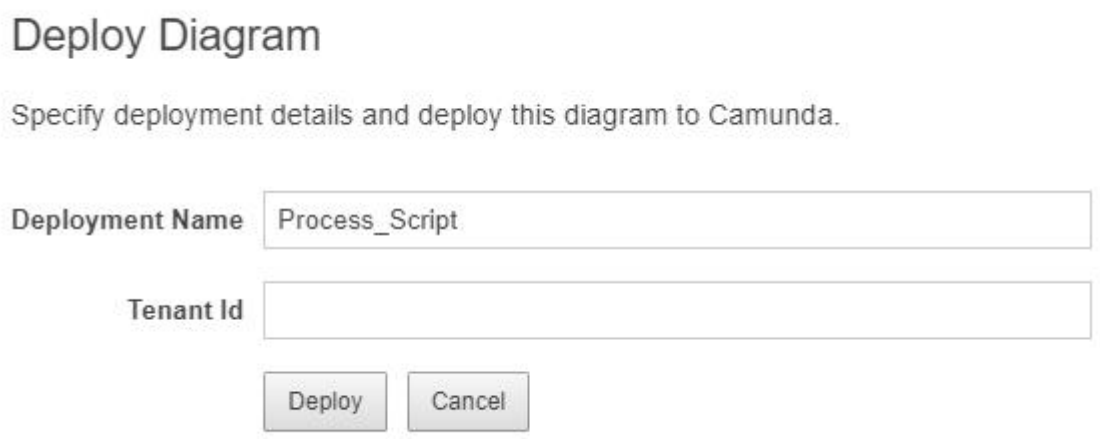
Script Format: groovy

Script Type: Inline Script

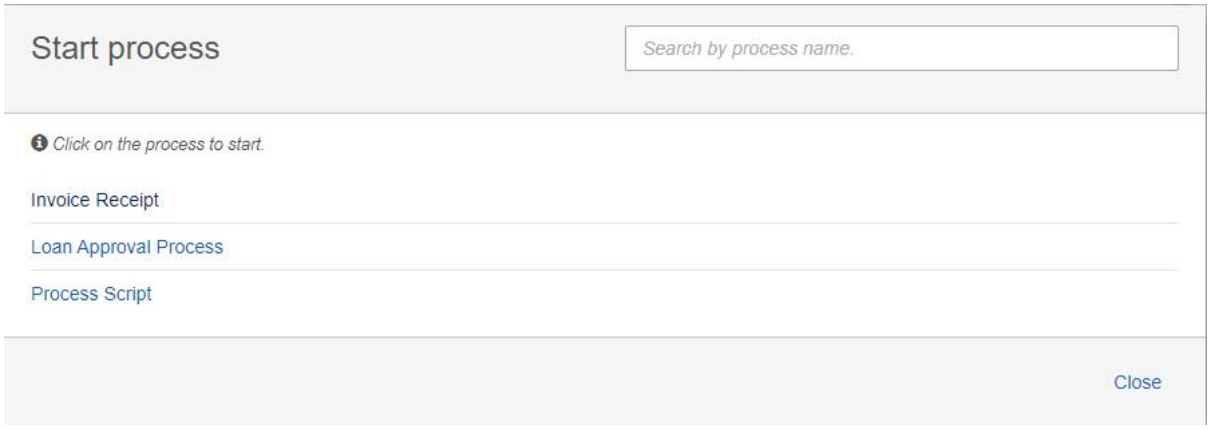
Script:

```
println "Wow, Bank wins"
```

Deploy the process through Camunda modeler.



Start the deployed process multiple times from Camunda Tasklist application.



Check the console output:

Wow, Glamber wins

Wow, Glamber wins

Wow, Glamber wins

Wow, Bank wins

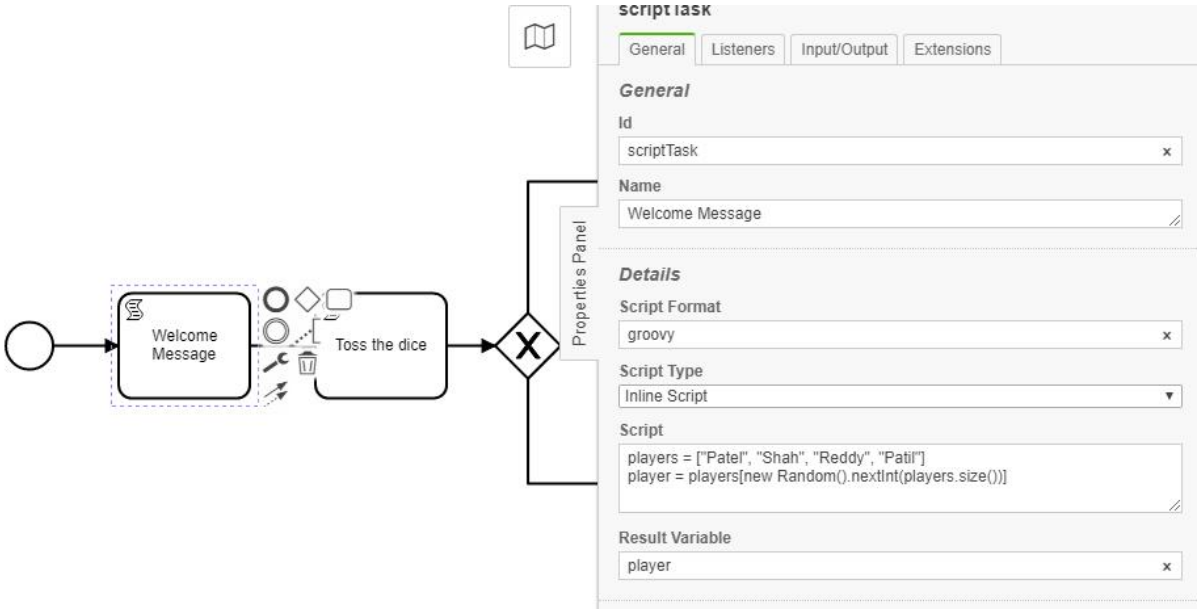
Wow, Bank wins

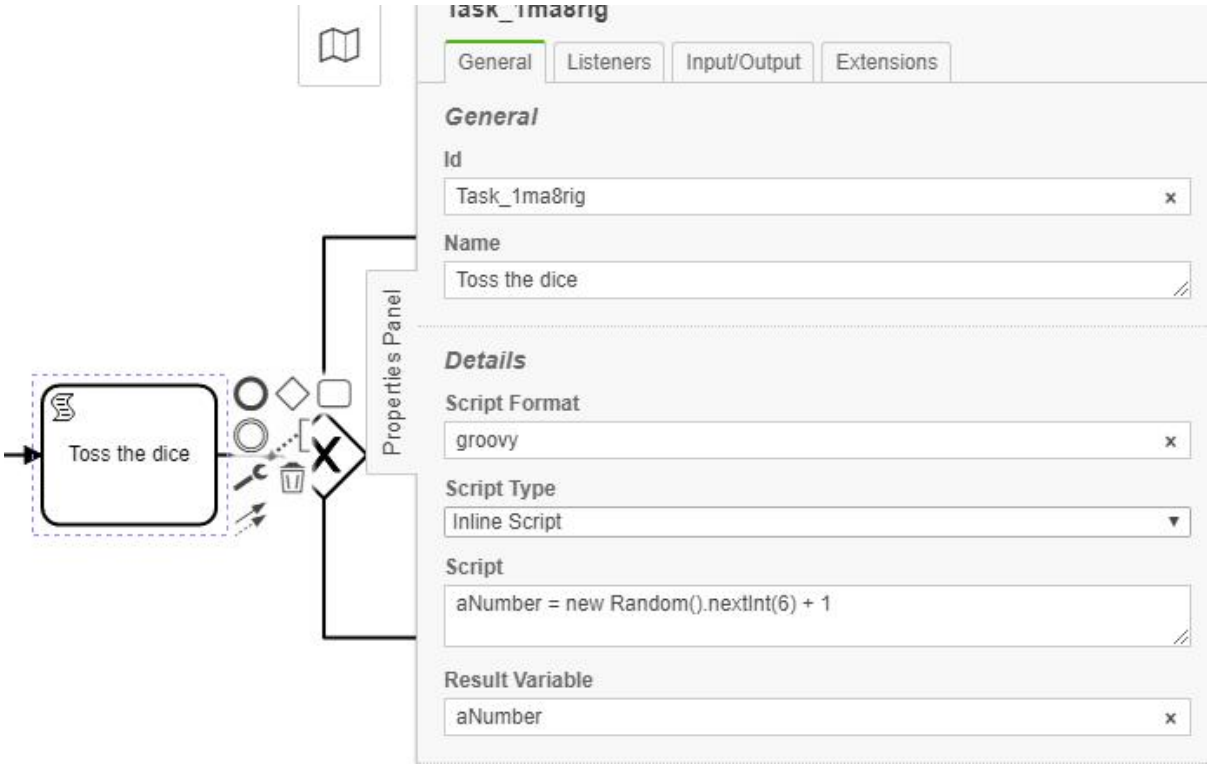
Wow, Glamber wins

Wow, Glamber wins

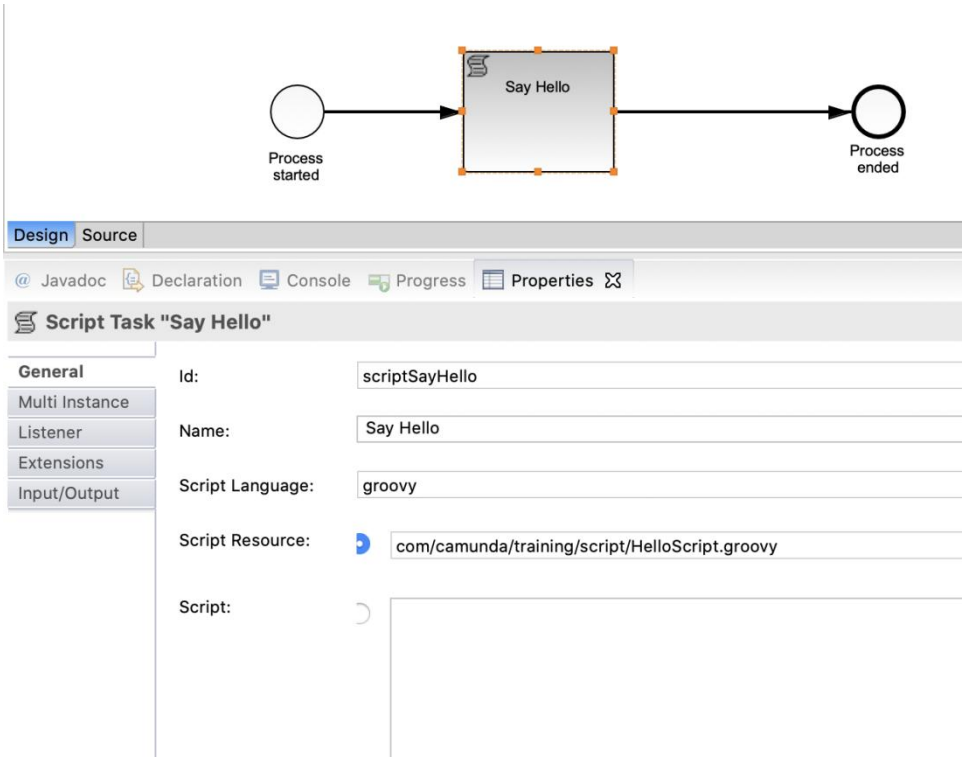
Script Result

The return value of a Script Task can be assigned to a previously existing or to a new process variable by specifying the process variable name as a literal value for the camunda:resultVariable attribute of a Script Task definition. Any existing value for a specific process variable will be overwritten by the result value of the script execution. When a result variable name is not specified, the script result value gets ignored.

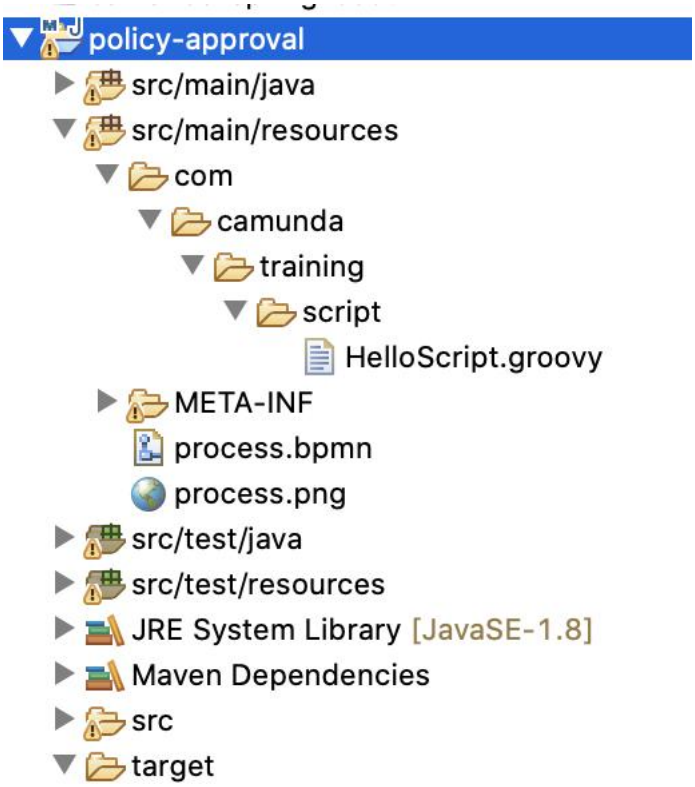




So far we have used inline script. Now let's use external script.



Create HelloScript.geoovy in resources/com/camunda/training/script directory.



Where to uses script:

Use Scripts as Execution Listeners

Use Script as Task listeners

Use Script as Conditions

Use Script as InputOutput parameters

Variables Available During Script Execution

During the execution of scripts, all process variables visible in the current scope are available. They can be accessed directly by the name of the variable (i.e., sum).

There are also special variables:

1. execution, which is always available if the script is executed in an execution scope (e.g., in a script task) (DelegateExecution).
2. task, which is available if the script is executed in a task scope (e.g., a task listener) (DelegateTask).

Execution Variables:

execution.activityInstanceId
execution.businessKey
execution.currentActivityId
execution.currentActivityName
execution.currentTransitionId
execution.eventName
execution.id
execution.processBusinessKey
execution.processDefinitionId
execution.processInstanceId
execution.tenantId

Execution methods:
execution.setVariable(variableName, value);
execution.getVariable(variableName)

Script:

Script format: groovy

Script:

```
out.println "Execution instance id : "+execution.activityInstanceId
out.println "Business key : "+execution.businessKey
out.println "Current activity id : "+execution.currentActivityId
out.println "Current activity name : "+execution.currentActivityName
out.println "Current transition id : "+execution.currentTransitionId
out.println "Event name : "+execution.eventName
out.println "Execution id : "+execution.id
out.println "Process business key : "+execution.processBusinessKey
out.println "Process definition id : "+execution.processDefinitionId
out.println "Process instance id : "+execution.processInstanceId
```

```
out.println "Tenant id : "+execution.tenantId
```

Task Variables:

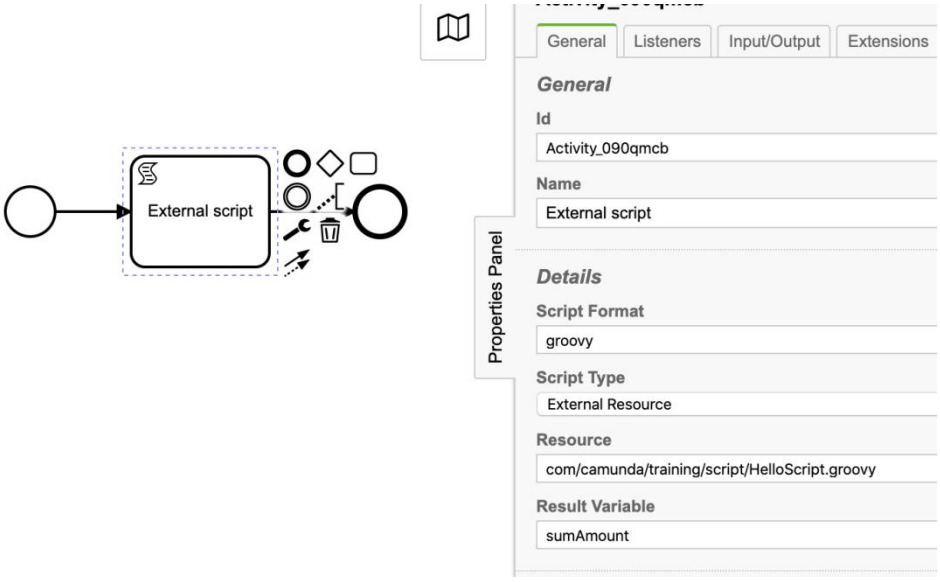
task.assignee
task.caseExecutionId
task.caseDefinitionId
task.caseInstanceId
task.caseDefinitionId
task.createTime
task.deleteReason
task.description
task.dueDate
task.eventName
task.executionId
task.followUpDate
task.id
task.name
task.owner
task.priority
task.processDefinitionId
task.processInstanceId
task.taskDefinitionKey
task.tenantId

Task methods:

task.setVariable(variableName, value);
task.getVariable(variableName)

External Script

The script source code can also be loaded from an external resource.



Copy

HelloScript.groovy to camunda-bpm-tomcat-7.14.0/server/apache-tomcat-9.0.36/webapps/camunda/WEB-INF/classes/com/camunda/script/HelloScript.groovy

HelloScript.groovy

```
println "External Script called";
number1 = execution.getVariable("number1");
number2 = execution.getVariable("number2");

totalAmount = number1 + number2
```